

รายงานสัปดาห์ที่ 8 (Advanced)

Agentic Data Processing — Advanced CSV & JSON Workflows

สรุปเนื้อหา บันทึกผลแล็บ การวิเคราะห์และแก้บั๊ก 6 จุด พร้อมผลการรันจริง
โปรเจกต์: agentic-data-processor

1. ผลลัพธ์การเรียนรู้ (Learning Outcomes)

- ออกแบบและสร้าง "Data Agent" ที่ทำงานตาม configuration และรันงานหลายขั้นตอนต่อเนื่องกันได้
- ใช้เทคนิคจัดการข้อมูล CSV ขั้นสูง ได้แก่ การจัดกลุ่ม (grouping) การรวมยอด (aggregation) และการกรอง
- ใช้เทคนิคค้นหาและปรับโครงสร้างข้อมูล JSON ที่ซับซ้อน รวมถึงการอัปเดตและเพิ่มข้อมูล
- แปลงข้อมูลข้ามรูปแบบระหว่าง CSV และ JSON ภายใน workflow อัตโนมัติ
- จัดการสถานะ (state) ภายใน agent เพื่อส่งข้อมูลต่อระหว่าง task
- ประยุกต์หลักการออกแบบแบบโมดูล การจัดการข้อผิดพลาด และการทำ logging สำหรับ data pipeline

2. สรุปเนื้อหาบทเรียน

2.1 แนวคิด Agentic สำหรับการประมวลผลข้อมูล

แทนที่จะเขียนโค้ดเรียกฟังก์ชันทีละขั้นแบบตายตัว เราแยก "ตรรกะการทำงาน" (โค้ด) ออกจาก "ลำดับงานที่ต้องทำ" (config) โดยนิยาม pipeline ไว้ในไฟล์ JSON แล้วให้ agent อ่านและทำตาม ข้อดีคือเปลี่ยนลำดับงาน เพิ่ม/ลด ขั้นตอน หรือเปลี่ยนไฟล์เป้าหมายได้โดยไม่ต้องแก้โค้ดเลย และหัวใจสำคัญคือ data store ซึ่งเป็น dictionary ภายใน agent ที่ใช้เก็บผลลัพธ์ระหว่างทาง ทำให้ task ถัดไปหยิบผลลัพธ์ของ task ก่อนหน้ามาใช้ต่อได้ผ่าน input_data_key และ output_data_key

2.2 การจัดการ CSV ขั้นสูง

- Aggregation: จัดกลุ่มข้อมูลตามคอลัมน์ (เช่น Customer) แล้วรวมยอด หานับจำนวน หรือหาค่าเฉลี่ย โดยใช้ defaultdict เก็บผลสะสมของแต่ละกลุ่ม
- Data Cleaning: แปลงชนิดข้อมูลอย่างปลอดภัยด้วย try-except และข้ามแถวที่ข้อมูลผิดรูปแบบ
- Filtering: กรองข้อมูลด้วยตัวดำเนินการที่กำหนดใน config (>, <, >=, <=, ==, !=)

2.3 การจัดการ JSON ขั้นสูง

- Dot-notation path: อ้างถึงข้อมูลที่ซับซ้อนด้วยเส้นทางแบบจุด เช่น 0.details.brand หมายถึงสมาชิกลำดับที่ 0 → คีย์ details → คีย์ brand
- Operations: รองรับ set (กำหนดค่า), add_to_list (เพิ่มสมาชิก) และ remove_from_list (ลบสมาชิก)
- Deep copy: ใช้ json.loads(json.dumps(data)) คัดลอกข้อมูลก่อนแก้ไข เพื่อไม่ให้กระทบข้อมูลต้นฉบับใน data store

- Query: ดึงค่าเฉพาะจุดจากข้อมูลซ้อนชั้นโดยไม่ต้องเขียนโค้ดไล่ key เอง

2.4 การแปลงข้ามรูปแบบและ Audit Logging

การแปลง CSV → JSON ไม่ใช่แค่เปลี่ยนรูปแบบไฟล์ แต่ต้องแปลงชนิดข้อมูลด้วย เพราะ CSV เก็บทุกอย่างเป็น string ฟังก์ชัน `csv_to_json()` จึงตรวจสอบและแปลงเป็น int, float, boolean หรือ None ให้อัตโนมัติ ส่วน Audit Log จะบันทึกทุกขั้นตอนพร้อม timestamp และสถานะ (INFO / SUCCESS / ERROR) ลงไฟล์ JSON ทำให้ตรวจสอบย้อนหลังได้ว่า task ไหนสำเร็จหรือล้มเหลว ซึ่งจำเป็นมากเมื่อ pipeline มีหลายขั้นตอน

3. สรุปผลการทำแล็บ

Pipeline ที่กำหนดใน `configs/data_pipeline_config.json` มีทั้งหมด 9 ขั้นตอน แต่ละขั้นส่งผลลัพธ์ต่อกันผ่าน data store ดังนี้

#	Task	input_data_key	output_data_key
1	Load Sales Data CSV	—	raw_sales_data
2	Aggregate Sales by Customer	raw_sales_data	customer_sales_summary
3	Save Aggregated Sales CSV	customer_sales_summary	— (เขียนไฟล์)
4	Convert Raw Sales to JSON	raw_sales_data	raw_sales_json
5	Save Raw Sales JSON	raw_sales_json	— (เขียนไฟล์)
6	Load Inventory Data JSON	—	raw_inventory_data
7	Update Inventory & Add Product	raw_inventory_data	updated_inventory_data
8	Query Product Detail	updated_inventory_data	queried_product_brand
9	Save Updated Inventory JSON	updated_inventory_data	— (เขียนไฟล์)

4. บั๊กที่พบในโค้ดต้นฉบับและการแก้ไข

เมื่อนำโค้ดจากเอกสารประกอบการสอนมาสร้างเป็นโปรเจกต์จริงแล้วรัน พบข้อผิดพลาดรวม 6 จุด โดย 3 จุดแรกทำให้โปรแกรมรันไม่ผ่านเลย ส่วนอีก 3 จุดทำให้โปรแกรมรันได้แต่ทำงานไม่ครบตามที่โจทย์กำหนด ซึ่งอันตรายกว่าเพราะไม่แสดง error ชัดเจน

4.1 ข้อผิดพลาดที่พบจากการรันจริง

```
week8/advanced — ข้อผิดพลาดก่อนแก้ไข

PS C:\xampp\htdocs\proj\week8\advanced> python main.py

# ---- รอบที่ 1: รันโค้ดตามต้นฉบับ ----
Traceback (most recent call last):
  File "main.py", line 8, in <module>
    from config_parser import ConfigParser
  File "src\config_parser.py", line 5, in <module>
    from .utils import setup_logging
ImportError: attempted relative import with no known parent package

# ---- รอบที่ 2: หลังแก้ import แล้ว ----
config_parser - CRITICAL - An error occurred while loading config:
  Task 2 (type: save_csv) is missing 'output_data_key' field.
ValueError: Task 2 (type: save_csv) is missing 'output_data_key' field.

# ---- รอบที่ 3: หลังแก้ validator แล้ว ----
json_tasks - WARNING - Update skipped: missing 'path' in update:
  {'path': '', 'value': {'id': 'PROD004', 'name': 'Smart Speaker', ...},
  'operation': 'add_to_list'}
json_tasks - WARNING - Path 'PROD001.details.color' not navigable at key
  'PROD001' (expected dict/list, got <class 'list'>). Skipping update
__main__ - CRITICAL - An unexpected critical error occurred during execution:
  name 'sys' is not defined
NameError: name 'sys' is not defined. Did you forget to import 'sys'?
```

ภาพที่ 1: ข้อผิดพลาดที่พบในการรันแต่ละรอบ ก่อนแก้ไข

4.2 ตารางสรุปบักทั้ง 6 จุด

#	ไฟล์	อาการ	สาเหตุและการแก้ไข
1	src/*.py (5 ไฟล์)	ImportError: attempted relative import with no known parent package	main.py ใช้วิธี sys.path.append('src') ทำให้โมดูลถูกโหลดเป็น top-level ไม่ใช่ package จึงใช้ relative import ไม่ได้ → เปลี่ยนเป็น absolute import
2	config_parser.py	pipeline หยุดที่ task 2: missing 'output_data_key'	validator บังคับให้ทุก task มี output_data_key แต่ task save_* เป็นปลายทางที่ไม่ผลิตข้อมูล → แยกเงื่อนไขให้ save_* ต้องการแค่ input_data_key
3	data_agent.py	NameError: name 'sys' is not defined	โค้ดเรียก sys.modules ตอนสร้าง audit report แต่ไม่ได้ import sys → เพิ่ม import sys
4	json_tasks.py	สินค้าใหม่ PROD004 ไม่ถูกเพิ่ม (เตือนว่า missing 'path')	config ตั้งใจใช้ path วางเพื่อเพิ่มสมาชิกเข้า list ราก แต่โค้ดมองว่าเป็นค่าว่างแล้วข้าม → เพิ่มเงื่อนไขรองรับ add_to_list ที่ราก
5	data_pipeline_config.json	path 'PROD001.details.color' ใช้ไม่ได้	ข้อมูลสินค้าขึ้นนอกเป็น list ไม่ใช่ dict ที่มีคีย์ชื่อ PROD001 → เปลี่ยน path เป็น 0.details.color ให้สอดคล้องกับ update อื่น
6	utils.py	log ถูกพิมพ์ซ้ำ 2 บรรทัด (99 บรรทัด)	basicConfig ใส่ handler ที่ root แล้ว setup_logging ใส่อีกที่ logger ลูก เมื่อ propagate ขึ้น root จึงพิมพ์ซ้ำ → เพิ่ม logger.propagate = False

4.3 โค้ดที่แก้ไขทั้งหมด

บ๊ิก 1: src/*.py (5 ไฟล์) — เปลี่ยน relative import เป็น absolute - from .utils import setup_logging + from utils import ensure_directory_exists, setup_logging
บ๊ิก 2: src/config_parser.py — ยกเว้น task save_* จากการบังคับ output_data_key - if "output_data_key" not in task: - raise ValueError(f"Task {i} ... missing 'output_data_key'") + if task["type"] in ["save_csv", "save_json"]: + if "input_data_key" not in task: + raise ValueError(f"Task {i} ... requires 'input_data_key'.") + elif "output_data_key" not in task: + raise ValueError(f"Task {i} ... missing 'output_data_key'")
บ๊ิก 3: src/data_agent.py — เพิ่ม import ที่ขาดไป import os + import sys
บ๊ิก 4: src/json_tasks.py — รองรับ add_to_list ที่ราก (path ว่าง) + if operation == "add_to_list" and not path: + if isinstance(modified_data, list): + modified_data.append(value) + logger.info(f"Added new item to root list: {label}") + continue if not path: logger.warning(...) # เต็มศึกทุกกรณีเงิน add_to_list ถูกข้าม
บ๊ิก 5: configs/data_pipeline_config.json — แก่ path ให้อ้างด้วย index - {"path": "PROD001.details.color", "value": "Space Grey", ...} + {"path": "0.details.color", "value": "Space Grey", ...}
บ๊ิก 6: src/utils.py — หยุด log ขำ logger.setLevel(level) + logger.propagate = False

ภาพที่ 2: สรุปการแก้ไขทั้ง 6 จุด — สีแดงคือบรรทัดที่ลบออก สีเขียวคือบรรทัดที่เพิ่มเข้าไป

5. ผลการรันจริงหลังแก้ไข (Test Results)

หลังแก้ครบทั้ง 6 จุด โปรแกรมทำงานจบสมบูรณ์ (exit code 0) ทุก task รายงานสถานะ SUCCESS ไม่มี error หรือ warning เหลืออยู่เลย และจำนวนบรรทัด log ลดจาก 99 เหลือ 64 บรรทัด

```
week8/advanced — ผลการรันหลังแก้ไขครบทุกจุด

PS C:\xampp\htdocs\proj\week8\advanced> python main.py
2026-09-08 11:29:32,646 - INFO - Directory ensured: .data
2026-09-08 11:29:32,646 - INFO - Directory ensured: .configs
2026-09-08 11:29:32,646 - INFO - Directory ensured: .reports
2026-09-08 11:29:32,646 - INFO - Loading configuration from: .\configs\data_pipeline_config.json
2026-09-08 11:29:32,646 - INFO - Configuration validated.
2026-09-08 11:29:32,646 - INFO - Configuration loaded successfully from .\configs\data_pipeline_config.json
2026-09-08 11:29:32,646 - INFO - Initializing Data Agent...
2026-09-08 11:29:32,647 - INFO - CSVTasks initialized.
2026-09-08 11:29:32,647 - INFO - JSONTasks initialized.
2026-09-08 11:29:32,647 - INFO - ConversionTasks initialized.
2026-09-08 11:29:32,647 - INFO - Data Agent initialized with configuration.
2026-09-08 11:29:32,647 - INFO - Running Data Agent workflow...
2026-09-08 11:29:32,647 - INFO - Starting Data Agent run...
2026-09-08 11:29:32,647 - INFO - AUDIT LOG - INFO: Data Agent started. | Task: agent_run
2026-09-08 11:29:32,647 - INFO - Executing task: 'Load Sales Data CSV' (Type: load_csv)
2026-09-08 11:29:32,647 - INFO - AUDIT LOG - INFO: Starting task 'Load Sales Data CSV' | Task: load_csv
2026-09-08 11:29:32,647 - INFO - Successfully loaded 10 rows from '.data/input_sales.csv'.
2026-09-08 11:29:32,647 - INFO - AUDIT LOG - SUCCESS: Task 'Load Sales Data CSV' completed successfully. | Task: load_csv
2026-09-08 11:29:32,647 - INFO - Executing task: 'Aggregate Sales by Customer' (Type: transform_csv_aggregate)
2026-09-08 11:29:32,647 - INFO - AUDIT LOG - INFO: Starting task 'Aggregate Sales by Customer' | Task: transform_csv_aggregate
2026-09-08 11:29:32,647 - INFO - Aggregated data by 'Customer' using 'sum'. Produced 5 rows.
2026-09-08 11:29:32,647 - INFO - AUDIT LOG - SUCCESS: Task 'Aggregate Sales by Customer' completed successfully. | Task: transform_csv_aggregate
2026-09-08 11:29:32,647 - INFO - Executing task: 'Save Aggregated Sales CSV' (Type: save_csv)
2026-09-08 11:29:32,647 - INFO - AUDIT LOG - INFO: Starting task 'Save Aggregated Sales CSV' | Task: save_csv
2026-09-08 11:29:32,647 - INFO - Directory ensured: .data
2026-09-08 11:29:32,648 - INFO - Successfully saved 5 rows to '.data/processed_sales_by_customer.csv'.
2026-09-08 11:29:32,648 - INFO - AUDIT LOG - SUCCESS: Task 'Save Aggregated Sales CSV' completed successfully. | Task: save_csv
2026-09-08 11:29:32,648 - INFO - Executing task: 'Convert Raw Sales to JSON' (Type: csv_to_json_conversion)
2026-09-08 11:29:32,648 - INFO - AUDIT LOG - INFO: Starting task 'Convert Raw Sales to JSON' | Task: csv_to_json_conversion
2026-09-08 11:29:32,648 - INFO - Converted 10 CSV rows to JSON format.
2026-09-08 11:29:32,648 - INFO - AUDIT LOG - SUCCESS: Task 'Convert Raw Sales to JSON' completed successfully. | Task: csv_to_json_conversion
2026-09-08 11:29:32,648 - INFO - Executing task: 'Save Raw Sales JSON' (Type: save_json)
2026-09-08 11:29:32,648 - INFO - AUDIT LOG - INFO: Starting task 'Save Raw Sales JSON' | Task: save_json
2026-09-08 11:29:32,648 - INFO - Directory ensured: .data
2026-09-08 11:29:32,648 - INFO - Successfully saved JSON to '.data/sales_data.json'.
2026-09-08 11:29:32,648 - INFO - AUDIT LOG - SUCCESS: Task 'Save Raw Sales JSON' completed successfully. | Task: save_json
2026-09-08 11:29:32,648 - INFO - Executing task: 'Load Inventory Data JSON' (Type: load_json)
2026-09-08 11:29:32,648 - INFO - AUDIT LOG - INFO: Starting task 'Load Inventory Data JSON' | Task: load_json
2026-09-08 11:29:32,649 - INFO - Successfully loaded JSON from '.data/input_inventory.json'.
2026-09-08 11:29:32,649 - INFO - AUDIT LOG - SUCCESS: Task 'Load Inventory Data JSON' completed successfully. | Task: load_json
2026-09-08 11:29:32,649 - INFO - Executing task: 'Update Inventory Stock & Add Product' (Type: update_json_data)
2026-09-08 11:29:32,649 - INFO - AUDIT LOG - INFO: Starting task 'Update Inventory Stock & Add Product' | Task: update_json_data
2026-09-08 11:29:32,649 - INFO - Set '0.stock' to '75'.
2026-09-08 11:29:32,649 - INFO - Set '1.details.layout' to 'UK ISO'.
2026-09-08 11:29:32,649 - INFO - Set '2.details.dpi' to '2000'.
2026-09-08 11:29:32,649 - INFO - Added new item to root list: PROD004
2026-09-08 11:29:32,649 - INFO - Set '0.details.color' to 'Space Grey'.
2026-09-08 11:29:32,649 - INFO - AUDIT LOG - SUCCESS: Task 'Update Inventory Stock & Add Product' completed successfully. | Task: update_json_data
2026-09-08 11:29:32,649 - INFO - Executing task: 'Query Specific Product Detail' (Type: query_json_data)
2026-09-08 11:29:32,649 - INFO - AUDIT LOG - INFO: Starting task 'Query Specific Product Detail' | Task: query_json_data
2026-09-08 11:29:32,649 - INFO - Successfully queried '0.details.brand'. Result: TechCorp
2026-09-08 11:29:32,649 - INFO - AUDIT LOG - INFO: Query result for path '0.details.brand': TechCorp | Task: query_json_data
2026-09-08 11:29:32,650 - INFO - AUDIT LOG - SUCCESS: Task 'Query Specific Product Detail' completed successfully. | Task: query_json_data
2026-09-08 11:29:32,650 - INFO - Executing task: 'Save Updated Inventory JSON' (Type: save_json)
2026-09-08 11:29:32,650 - INFO - AUDIT LOG - INFO: Starting task 'Save Updated Inventory JSON' | Task: save_json
2026-09-08 11:29:32,650 - INFO - Directory ensured: .data
2026-09-08 11:29:32,651 - INFO - Successfully saved JSON to '.data/updated_inventory.json'.
2026-09-08 11:29:32,651 - INFO - AUDIT LOG - SUCCESS: Task 'Save Updated Inventory JSON' completed successfully. | Task: save_json
2026-09-08 11:29:32,651 - INFO - AUDIT LOG - INFO: Data Agent finished in 0.00 seconds. | Task: agent_run_summary
2026-09-08 11:29:32,651 - INFO - Data Agent finished. Total time: 0.00 seconds.
2026-09-08 11:29:32,651 - INFO - Directory ensured: .reports
2026-09-08 11:29:32,652 - INFO - Audit report saved to: .reports\audit_report_20260908_112932.json
2026-09-08 11:29:32,652 - INFO - AUDIT LOG - INFO: Audit report generated at .reports\audit_report_20260908_112932.json | Task: audit_report
2026-09-08 11:29:32,652 - INFO - Data automation process completed.
```

ภาพที่ 3: ผลการรันหลังแก้ไขครบ — ครบทั้ง 9 tasks สถานะ SUCCESS (ย่อเส้นทางเดิมเป็น "." เพื่อให้ภาพอ่านง่าย)

5.1 ผลลัพธ์การรวมยอดขายรายลูกค้า

Customer	Total_Amount	ที่มาของตัวเลข
Alice	1425.75	1200.50 + 75.25 + 150.00
Bob	130.75	25.00 + 45.75 + 60.00
Charlie	500.0	300.00 + 200.00
David	90.0	90.00
Eve	15.5	15.50

ตรวจสอบความถูกต้อง: ผลรวมทุกกลุ่ม = 1425.75 + 130.75 + 500.00 + 90.00 + 15.50 = 2162.00

ซึ่งเท่ากับยอดรวมของรายการขายทั้ง 10 รายการพอดี

5.2 ผลลัพธ์การแก้ไขข้อมูลสินค้าคงคลัง

คำสั่งใน config	ก่อน	หลัง	ผล
0.stock (set 75)	50	75	สำเร็จ

1.details.layout (set)	US ANSI	UK ISO	สำเร็จ
2.details.dpi (set)	1600	2000	สำเร็จ
add_to_list (PROD004)	ไม่มี	Smart Speaker	สำเร็จ (หลังแก้บั๊ก 4)
0.details.color (set)	ไม่มี	Space Grey	สำเร็จ (หลังแก้บั๊ก 5)
query 0.details.brand	—	TechCorp	สำเร็จ

6. คำถาม-คำตอบสำคัญ

6.1 การเขียนแบบ config-driven ดีกว่าเขียนโค้ดตรง ๆ อย่างไร?

ข้อดีหลักคือการแยกตรรกะออกจากลำดับงาน ถ้าต้องการเพิ่มขั้นตอน เปลี่ยนไฟล์เป้าหมาย หรือสลับลำดับงาน ก็แก้แค่ไฟล์ JSON โดยไม่ต้องแตะโค้ด Python เลย ทำให้คนที่ไม่ได้เขียนโปรแกรมก็ปรับ pipeline ได้ อีกทั้งโค้ดชุดเดียวยังนำไปใช้กับงานข้อมูลอื่นได้ทันทีเพียงเขียน config ใหม่ ซึ่งต่างจากการเขียนโค้ดตรง ๆ ที่ต้องแก้และทดสอบโปรแกรมใหม่ทุกครั้งที่ต้องการเปลี่ยน

6.2 data store สำคัญอย่างไรต่อการทำงานของ agent?

data store คือ dictionary ที่เก็บผลลัพธ์ระหว่างทางของแต่ละ task ทำให้ task ต่าง ๆ ส่งข้อมูลต่อกันได้โดยไม่ต้องเขียนไฟล์ลงดิสก์ทุกครั้ง เช่นในแล็บนี้ ข้อมูลขายดิบที่โหลดครั้งเดียว (raw_sales_data) ถูกนำไปใช้ซ้ำถึง 2 ที่ ทั้งการรวมยอดรายลูกค้าและการแปลงเป็น JSON ช่วยประหยัดทั้งเวลาและการอ่านไฟล์ซ้ำ

6.3 บทเรียนสำคัญจากการแก้บั๊กครั้งนี้คืออะไร?

บั๊กที่อันตรายที่สุดไม่ใช่บั๊กที่ทำให้โปรแกรมพัง แต่คือบั๊กที่ทำให้โปรแกรม "รันผ่าน" แต่ผลลัพธ์ไม่ครบ อย่างบั๊กที่ 4 และ 5 ที่แสดงเป็นเพียง WARNING แล้วข้ามไป ถ้าไม่ได้ตรวจไฟล์ผลลัพธ์เทียบกับสิ่งที่โจทย์กำหนด ก็จะไม่มีความรู้เลยว่าสินค้าใหม่ไม่เคยถูกเพิ่มเข้าไป บทเรียนคือต้องตรวจผลลัพธ์จริงเทียบกับสิ่งที่คาดหวังเสมอ ไม่ใช่ดูแค่ว่าโปรแกรมจบการทำงานโดยไม่ error

7. บทสรุป

แล็บนี้ยกระดับจากการเขียนสคริปต์ประมวลผลข้อมูลแบบตรงไปตรงมา ไปสู่การออกแบบระบบที่ซับซ้อนด้วย configuration ซึ่งเป็นรูปแบบที่ใช้จริงในงาน data engineering สิ่งที่ได้เรียนรู้มีทั้งด้านเทคนิค เช่น การจัดการข้อมูลซ้อนชั้น การรวมยอดข้อมูล การแปลงข้ามรูปแบบ และการทำ audit log และด้านกระบวนการทำงาน คือการนำโค้ดที่ได้รับมาทดสอบจริง วิเคราะห์หาสาเหตุของข้อผิดพลาดที่ละเอียด แก่ไข แล้วตรวจสอบผลลัพธ์ซ้ำจนมั่นใจว่าถูกต้องครบถ้วนตามที่โจทย์กำหนด

อ้างอิง: เอกสารประกอบการสอน "LabX ADV: Agentic Data Processing (Advanced CSV & JSON Workflows)"

หมายเหตุ: ภาพผลการรันสร้างจากข้อความ output จริงที่ได้จากการรันบนเครื่อง (ไม่ใช่ภาพจับหน้าจอ OS โดยตรง)
ค่าทั้งหมดในภาพเป็นผลลัพธ์จริงจากการรันจริง